

```

def __next__(self):
    if self.b < self.a:
        self.b = self.b + self.c
        return self.b
    else:
        raise StopIteration

if __name__ == "__main__":
    a1 = MyRange(3)
    print(next(a1))
    print(next(a1))

```

Let us look at the output:

```

>>> from myrange1 import MyRange
>>>
>>> m1 = MyRange(0,10,2)
>>>
>>> for each in m1:
...     print(each)
...
2
4
6
8
10

```

Let us see another output.

```

>>> m1 = MyRange(5)
>>> for each in m1:
...     print(each)
...
1
2
3
4

```

Let us list with it.

```

>>> m1 = MyRange(5)
>>> list(m1)
[1, 2, 3, 4, 5]
>>>

```

Now, you have probably understood the differences between iterable and iterator. Let us move on to the next topic.

Generator

A generator is a special kind of iterator. All generators are iterators but all iterators are not generators. Each time the *yield* statement is executed, the generator function generates a new value.

When a generator function gets called, then a generator object is returned without even beginning the execution of the

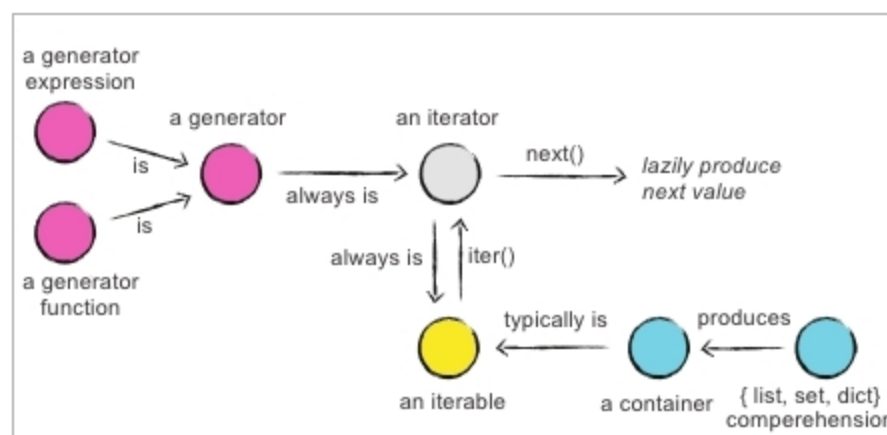


Figure 4: The relationship between the generator, iterable, iterator and container

generator function. When the *next()* function is called for the first time, the generator function starts executing until it reaches the *yield* statement. The yielded value is returned by the *next()* call.

Figure 4 shows that a generator or generator expression is always an iterator.

Now let us look at the code for a generator:

```

def myrange(a):
    i = 0
    while i < a:
        yield i
        i = i + 1

```

Let us see the output:

```

>>> from gene1 import myrange
>>> a = myrange(4)
>>> next(a)
0
>>> next(a)
1
>>> next(a)
2
>>> next(a)
3
>>> next(a)

```

Traceback (most recent call last):

```

File "<stdin>", line 1, in <module>
StopIteration

```

Generator expression

Let us understand the generator expression. The generator function is that in which the keyword *yield* appears in its body. The other type of generator is the generator expression, which is equivalent to a *list* comprehension. Its syntax is very similar to the *list* comprehension.

Let us look at the following series of examples.

```

>>> import sys
>>> list1 = [2,3,4,5,6]

```